

≡ Legendary Lapuras

水面模拟2.0

之前根据Minionsart的教程学习过一个卡通风格的[水面效果](#)。这次趁着作业打算学习一下稍微复杂一点的水面渲染。星云大大的[文章](#)总结得已经很全面了，之前写的那个shader大概就是正弦波和顶点高度位移图了，这次想试试Gerstner波实现。

水面波浪 Gerstner波

Gerstner波因为其计算量可控，性价比高，在游戏水体渲染领域应用较为广泛，比之前的正弦波而言，Gerstner波可以生成更为尖锐的波浪，波谷更为宽广，可以模拟粗犷的海面。关于波形的计算可以看[Catlike Coding-Wave](#)和[GPU Gems-Chap1](#)。

根据GPU Gems，Gerstner波的顶点计算公式可以表达为(看起来一大串其实都是加减乘除啦)：

$$P(x, y, t) = \begin{bmatrix} x + \sum (Q_i A_i \times D_i \cdot x \times \cos(w_i D_i \cdot (x, y) + \phi_i t)) \\ y + \sum (Q_i A_i \times D_i \cdot y \times \cos(w_i D_i \cdot (x, y) + \phi_i t)) \\ \sum (A_i \sin(w_i D_i \cdot (x, y) + \phi_i t)) \end{bmatrix}$$

其中，

$Q = \text{waveSteepness}$ ，当 $Q_i = 0$ ，即为普通sin波，要注意 Q_i 不得过大，否则波峰之上会出现诡异的面片交叉。

D 为方向向量， $j \cdot D$ 等于向量的 (x, y) 分量。

$A = \text{waveAmplitude}$ ，为波峰高度。

$w = \sqrt{\frac{2\pi}{\text{waveLength}}}$ ，为频率。

$\phi = \frac{\text{waveSpeed}}{\text{waveLength}}$ ，为相位。

法线部分的计算如下：

Legendary Lapuras

$$B = \begin{bmatrix} -\sum(Q_i \times D_i \cdot x \times D_i \cdot y \times w_i A_i \times S \\ \sum(D_i \cdot x \times w_i A_i \times C \end{bmatrix}$$

$$T = \begin{bmatrix} 1 - \sum(Q_i \times D_i \cdot x \times D_i \cdot y \times w_i A_i \times S \\ -\sum(Q_i \times D_i \cdot y^2 \times w_i A_i \times S \\ \sum(D_i \cdot y \times w_i A_i \times C \end{bmatrix}$$

其中,

$$S = \sin(w_i \times D_i \cdot P + \phi_i t)$$

$$C = \cos(w_i \times D_i \cdot P + \phi_i t)$$

根据以上公式得知，每个Gersner波需要输入的参数有——波纹方向、陡峭程度、波峰高度、波长、波纹速度。Shaderlab实现代码为：

```
float3 GerstnerWave(float4 wave, float3 p, float height, float steepness, inout float3 tange
    float wavelength = wave.z;
    float wavespeed = wave.w;
    float w = 1 / wavelength;
    float phase = wavespeed / wavelength;
    float2 d = normalize(wave.xy);
    float f = w * (dot(d, p.xz) + phase * _Time.y);

    steepness = clamp(0, wavelength / height, steepness);

    float qa = steepness * height;
    float wa = w * height;
    float qwa = w * qa;

    float3 displacement;
    displacement.x = d.x * qa * cos(f);
    displacement.z = d.y * qa * cos(f);
    displacement.y = height * sin(f);

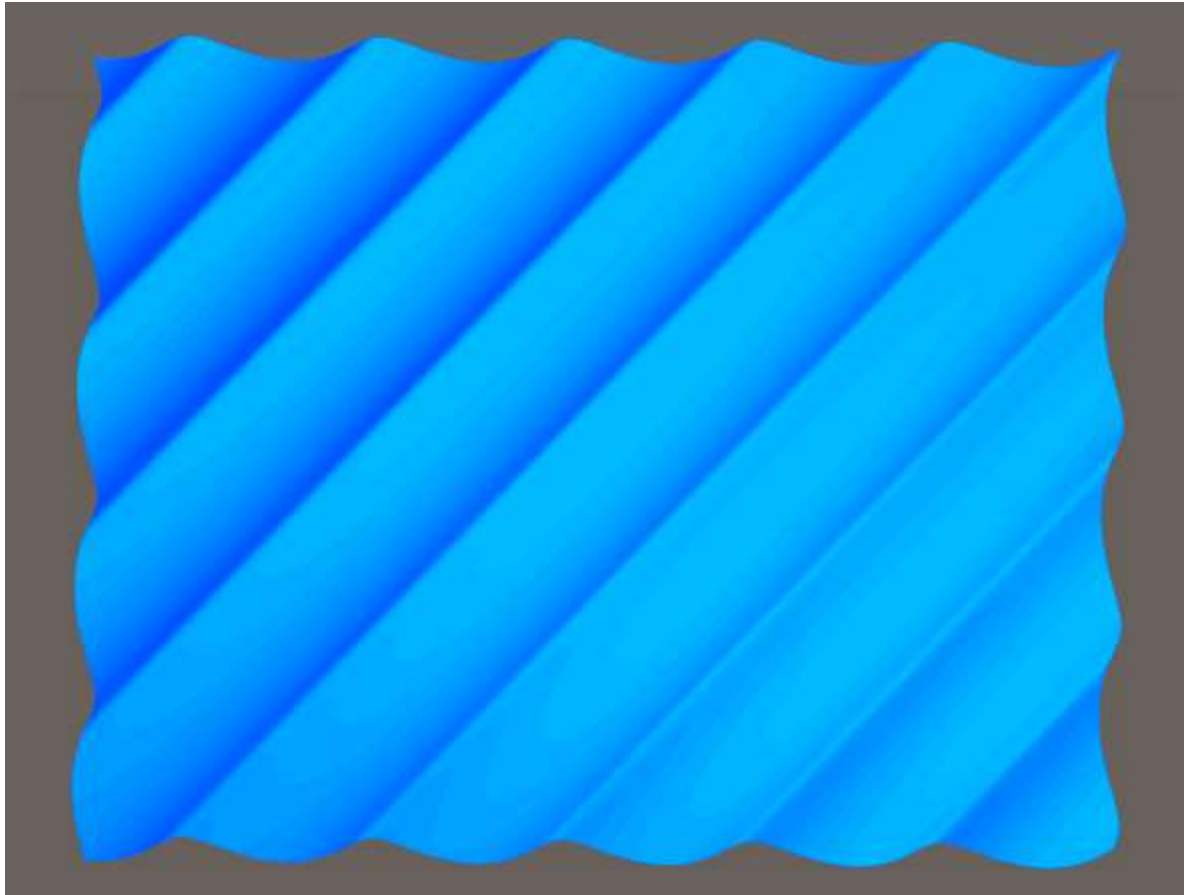
    tangent.x += -d.x * d.y * qwa * sin(f);
    tangent.y += d.y * wa * cos(f);
    tangent.z += -d.y * d.y * qwa * sin(f);

    binormal.x += -d.x * d.x * qwa * sin(f);
    binormal.y += d.x * wa * cos(f);
    binormal.z += -d.x * d.y * qwa * sin(f);
```

Legendary Lapuras

```
}
```

单个Gerstner波的动画如下：



通常会用多个Gerstner波叠加来模拟水面波纹。下图是使用三个Gerstner波生成的波浪。



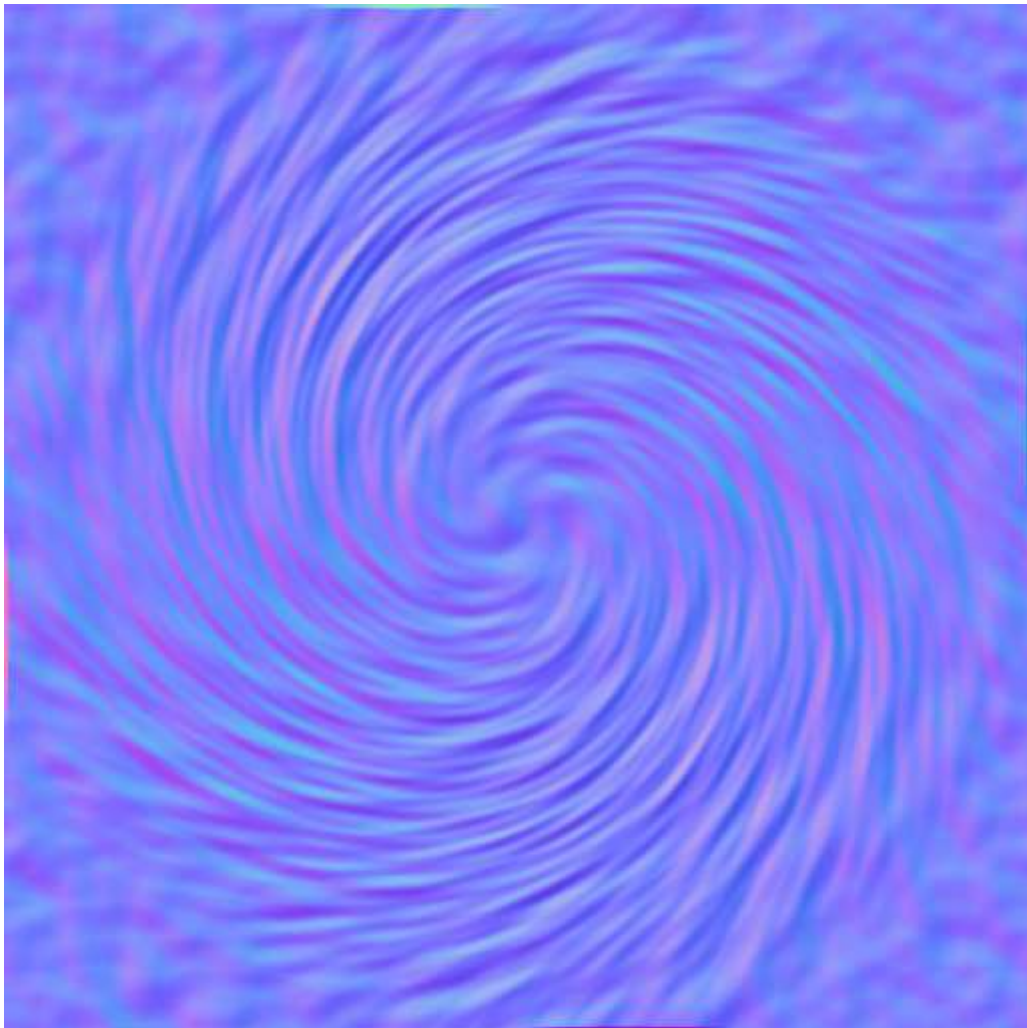
Legendary Lapuras

水面波光 Flow Map

之前做水面波光效果一般都是用一张或者几张法线叠加在一起。这次因为之后想做河流，所以添加了Flow Map来做这块。主要参考了[CatlikeCoding的教程](#)。

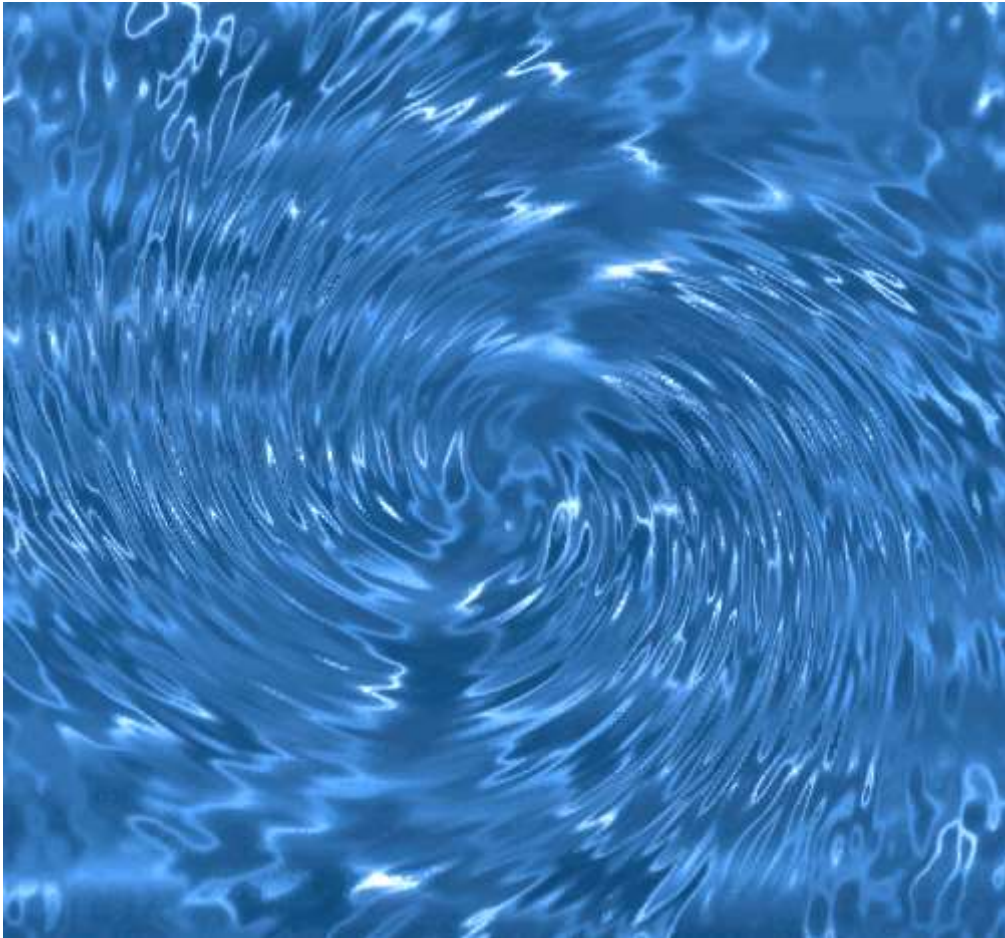
流型图（Flow Map），也常被称为矢量场图（Vector Field Map），本质上是一种基于矢量场平移法线贴图的着色技术，或者可以理解为一种UV动画，由VALVE的Vlachos在SIGGRAPH 2010上的talk [Water Flow in Portal 2](#)被大众所熟知。

以这张螺旋图作Flow Map为例。最终期望达到的效果是在主贴图基础上实现类似传送门的波纹动画。



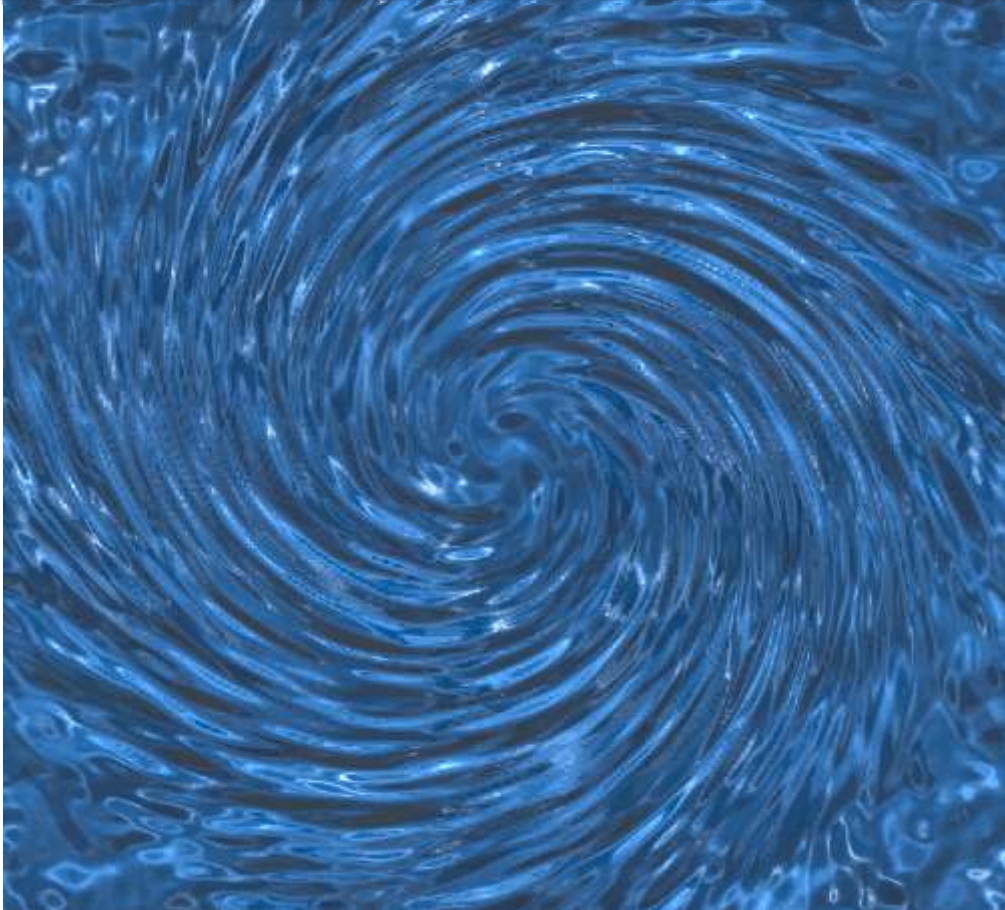
首先，借助从Flow Map中读取的颜色值作为扭曲的方向向量(只需要xz平面的二维向量)。根据方向向量对MainTex的uv进行位移，形成扭曲效果。这个循环是通过对时间取

Legendary Lapuras



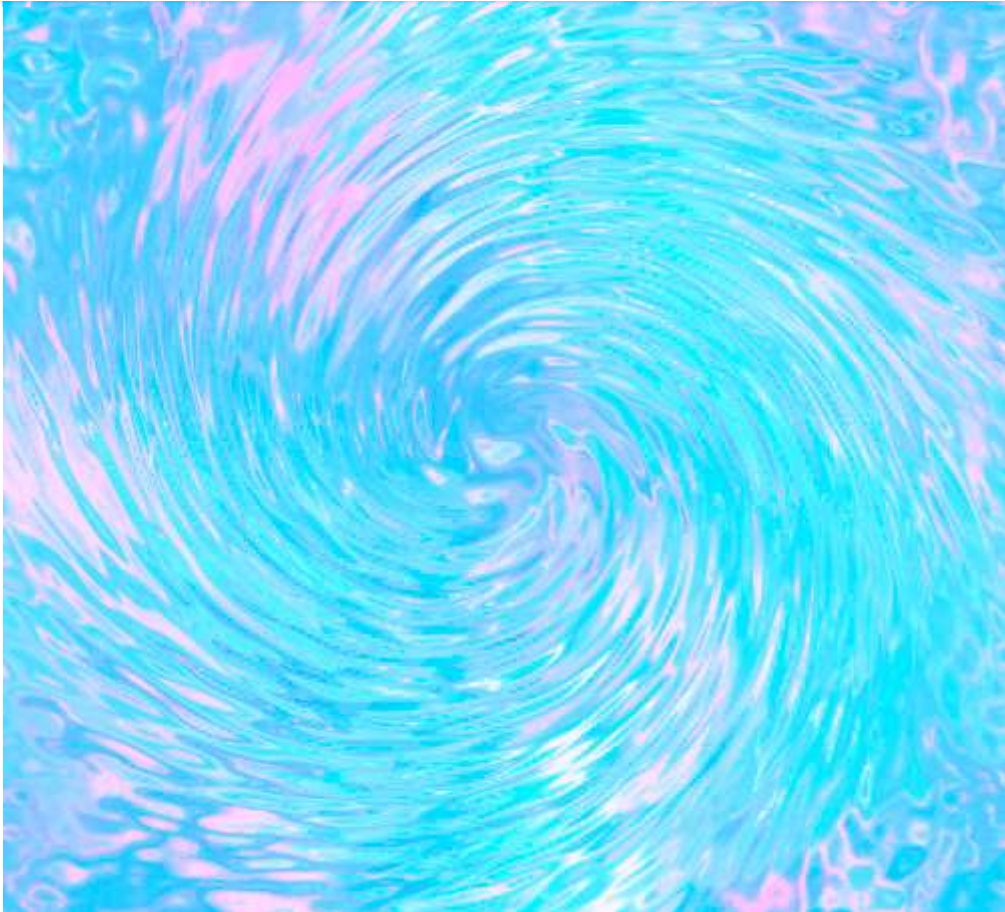
可以看见上图的循环基本呈“闪现”状态，要让过渡更加自然一些，可以加入一个平滑参数，对于这张图片而言，可以使用其Alpha通道达成此目的。

Legendary Lapuras

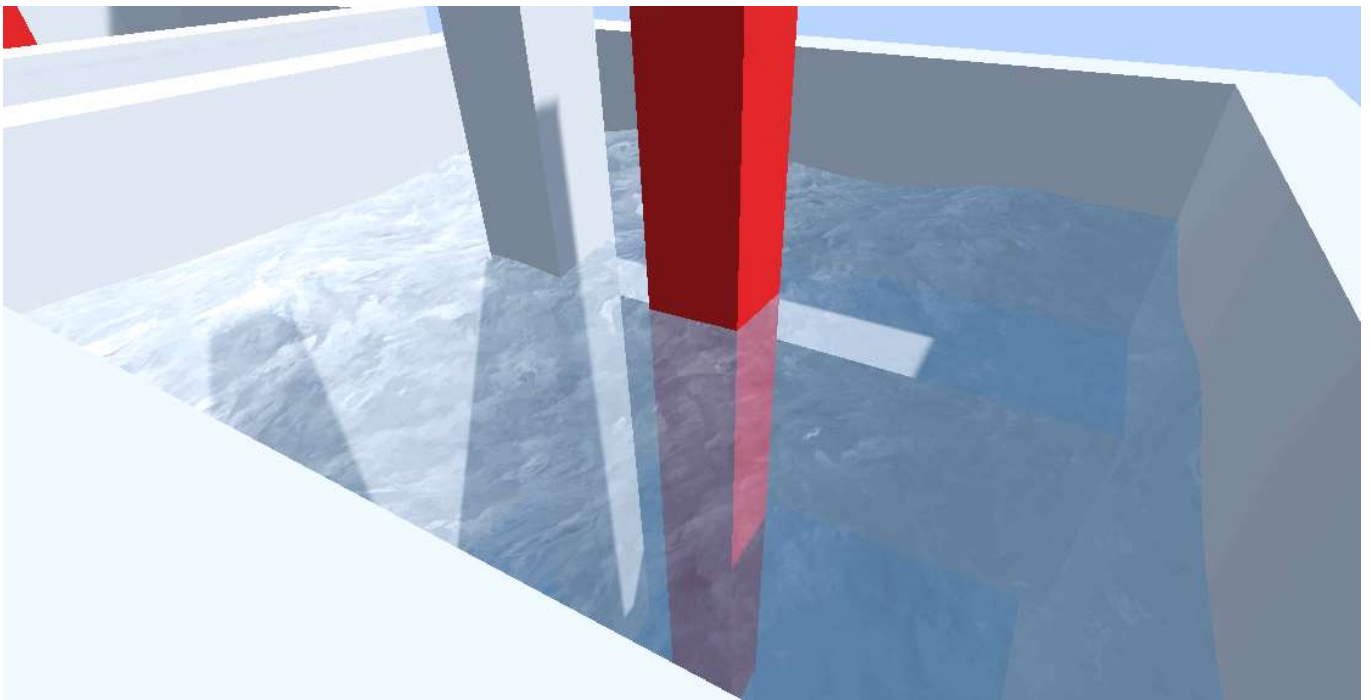


上图中，每个循环间的间隙依然很大，可以通过在时间错开并叠加两次来达成无缝衔接的目的。(呜呜，换了个颜色，是人鱼姬！)

Legendary Lapuras



做到这里就可以实现近似波光粼粼的效果了，想要的波纹形状和高光可以通过参数和贴图调整。



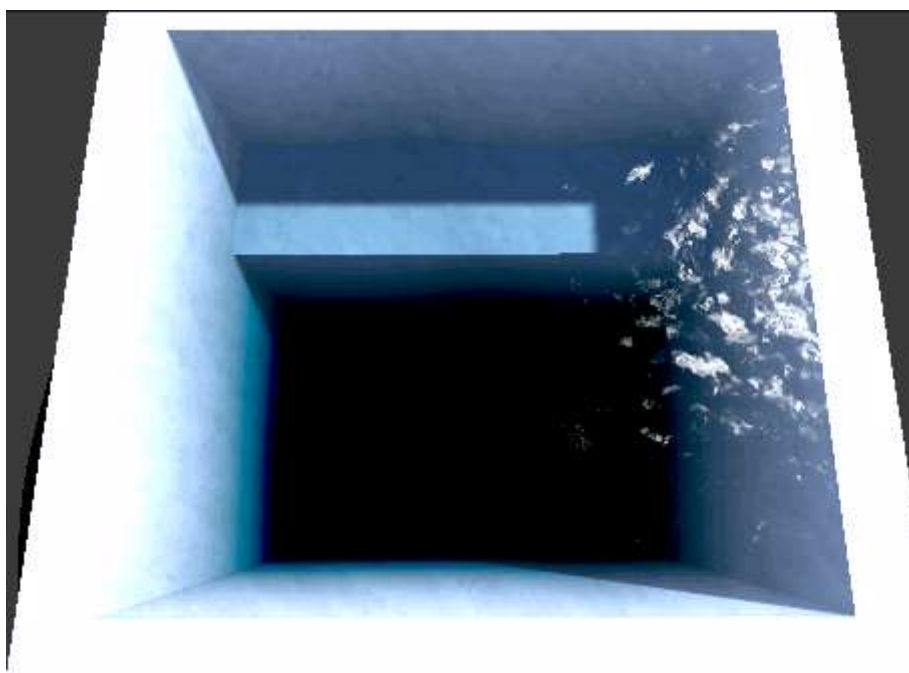
Legendary Lapuras

为透明。

这里我是根据Lux Water的启发，添加了*Underwater Fog*和*Light Absorption*两项。

*Underwater Fog*影响了水下可视度(透明度); *Light Absorption*则控制水对不同颜色光的吸收程度，换句话说决定了水下的颜色，通常水呈现偏蓝色，就是因为蓝色光被吸收得最少。

Light Absorption



水下颜色计算代码：

```
//underwater fog & color absorption
sampler2D _CameraDepthTexture, _WaterBackground; //_WaterBackground为GrabPass Texture
float4 _CameraDepthTexture_TexelSize;
float3 _WaterFogColor, _AbsorptionColor;
float _WaterFogDensity, _AbsorptionStrength, _AbsorptionDepth, _AbsorptionColorStrength;
float _Test;

float3 ColorBelowWater(float4 screenPos) {

    float2 uv = (((screenPos.xy) ) / screenPos.w);

    float backgroundDepth = LinearEyeDepth(SAMPLE_DEPTH_TEXTURE(_CameraDepthTexture, uv))
```


Legendary Lapuras

```
backgroundDepth = LinearEyeDepth(SAMPLE_DEPTH_TEXTURE(_CameraDepthTexture, uv));
depthDifference = backgroundDepth - surfaceDepth;
float3 backgroundColor = tex2D(_WaterBackground, uv).rgb;
float fogFactor = exp2(-_WaterFogDensity * depthDifference);

float3 ColorAbsorption = float3(1,1,1) - _AbsorptionColor;

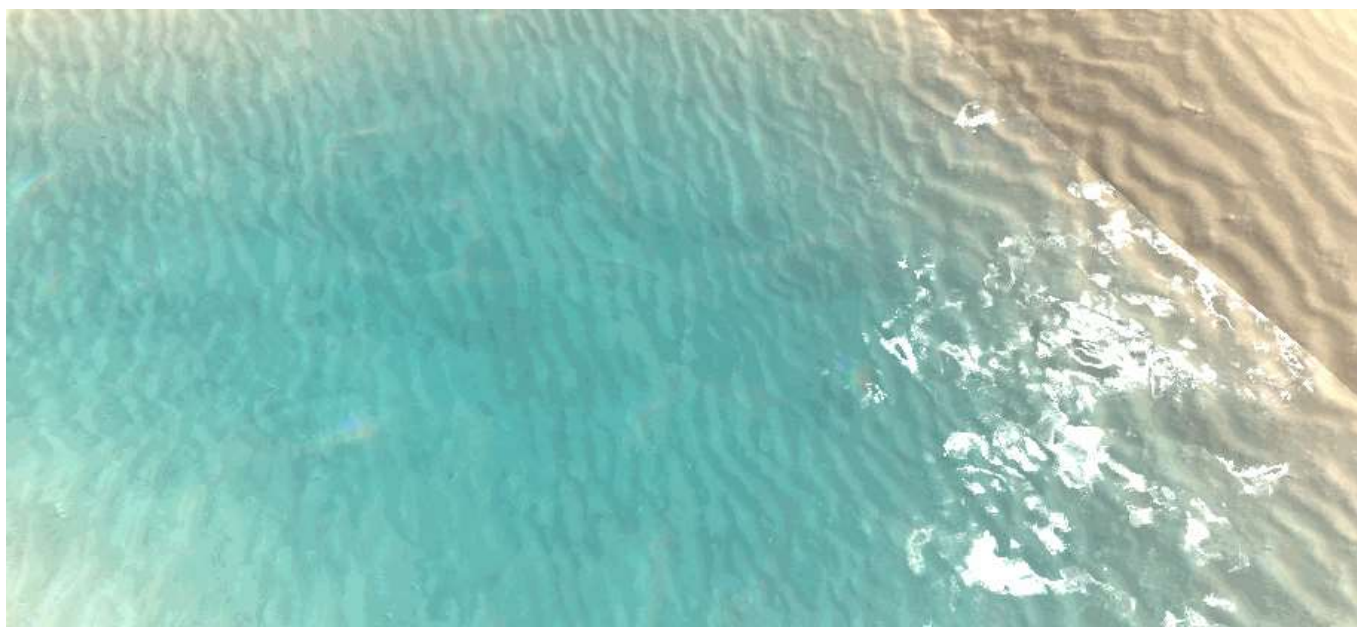
float d = exp2(-depthDifference * _AbsorptionDepth);
d = lerp(1, d, _AbsorptionStrength);
ColorAbsorption = lerp(d, -ColorAbsorption, _AbsorptionColorStrength * (1.0 - d));

return ColorAbsorption * lerp(_WaterFogColor, backgroundColor, fogFactor);
}
```

焦散 Caustics

之前用过Unity内置组件 `projector` 实现焦散，不过感觉在范围控制上并不是很方便，最好还是能跟水面shader放到一起嘛。

Alan Zucconi的这个[教程]给出了添加焦散贴图的方法，使用一张焦散贴图，在sample时用两套uv，uv的位移速度和tiling不同，然后叠加。另外还通过分离RGB通道形成虹光的效果，挺有趣的。



Legendary Lapuras

波动的影响(需要考虑法线)。

获取最近surface depth

```
float backgroundDepth = Linear01Depth(SAMPLE_DEPTH_TEXTURE(_CameraDepthTexture, uvscreen));

...
```c++
//caustics
float3 _SplitRGB;
sampler2D _CausticsTex;
float4 _Caustics1_ST, _Caustics2_ST;
float _Caustics1_Speed, _Caustics2_Speed, _CausticsTiling;

float3 CausticsColor(float2 mainuv) {
 //uv sets
 float2 uv = mainuv * _Caustics1_ST.xy + _Caustics1_ST.zw;
 float2 uv2 = mainuv * _Caustics2_ST.xy + _Caustics2_ST.zw;
 uv += _Caustics1_Speed * _Time.y;
 uv2 += _Caustics2_Speed * _Time.y;

 // RGB split
 float s1 = _SplitRGB;
 float r1 = tex2D(_CausticsTex, uv + float2(+s1, +s1)).r;
 float g1 = tex2D(_CausticsTex, uv + float2(+s1, -s1)).g;
 float b1 = tex2D(_CausticsTex, uv + float2(-s1, -s1)).b;

 float s2 = _SplitRGB;
 float r2 = tex2D(_CausticsTex, uv2 + float2(+s2, +s2)).r;
 float g2 = tex2D(_CausticsTex, uv2 + float2(+s2, -s2)).g;
 float b2 = tex2D(_CausticsTex, uv2 + float2(-s2, -s2)).b;

 float3 caustics1 = float3(r1, g1, b1);
 float3 caustics2 = float3(r2, g2, b2);

 float3 col = min(caustics1, caustics2);

 return col;
}
```

## 水岸混合

## 水面折射

## 水面反射

# Legendary Lapuras

Foam



Llapuras  by 阿尔芙咸